Matrix operations and FFT

Begin with block matrices and multiplication, proceed to elimination, PA=LU and least squares, then use the DFT and FFT for polynomial convolution.

Matrix methods organize computation through blocks and factorizations; FFT reuses roots of unity through even-odd splitting. Track dimensions, pivots, transform conventions and padding.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reuse the same coefficient matrix

In $Ax = b$, separate the coefficients from the right-hand side. Reuse a factorization when A stays fixed and b changes. Use real matrices, column vectors, $PA = LU$, and unit-diagonal L. Define the least-squares residual as $r = b - A\hat{x}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Match the inner dimensions

For $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$, $C = AB \in \mathbb{R}^{m \times n}$ with $c_{ij} = \sum_{t=1}^{k} a_{it} b_{tj}$. The direct algorithm uses mkn multiplications and $mn(k-1)$ additions. A defined AB does not imply a defined BA.

Matrix dimensions and one dot product

A: m × k

B: k × n

AB: m × n

Shared index t = 1...k

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compute entries with their indices

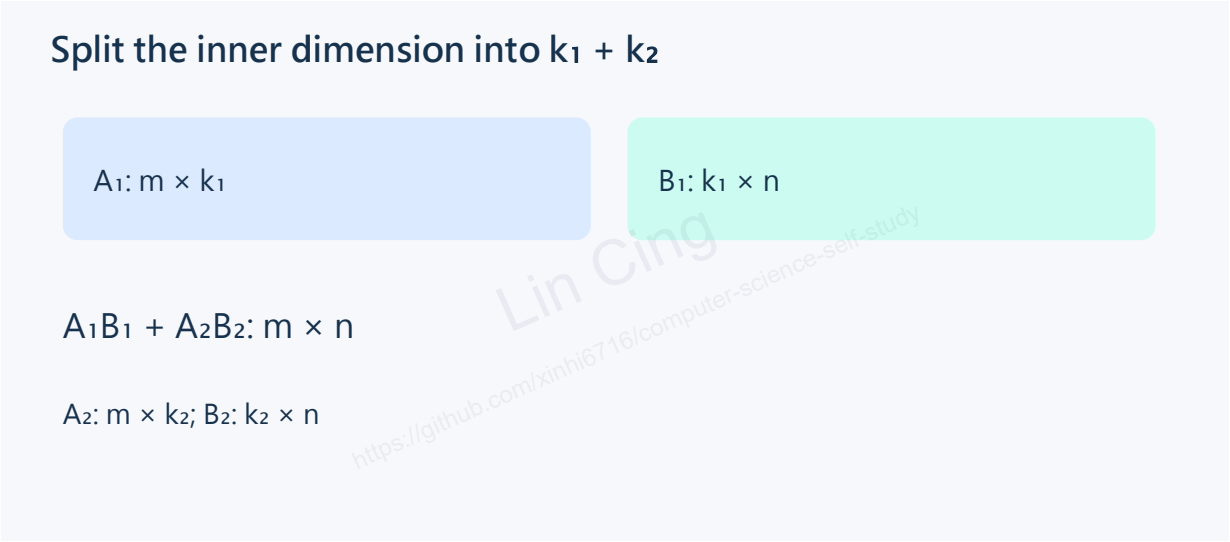
Let $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ and $B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$. Then $c_{12} = 1(0) + 2(1) + 3(1) = 5$ and $AB = \begin{bmatrix} 4 & 5 \\ 10 & 11 \end{bmatrix}$.

Here BA is 3×3, so it cannot equal AB.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Check dimensions inside every block

Split the inner dimension as $k = k_1 + k_2$. For $A = [A_1\ A_2]$ and $B = [B_1; B_2]$, let A_s have size $m \times k_s$ and B_s have size $k_s \times n$. Then $AB = A_1B_1 + A_2B_2$. In a two-by-two partition, $C_{ij} = A_{i1}B_{1j} + A_{i2}B_{2j}$. Blocks need not commute.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fewer recursive products change the exponent

Ordinary square blocking gives $8T(n/2) + \Theta(n^2) = \Theta(n^3)$. Strassen reconstructs four output blocks from seven products, giving $7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7})$. Pad with zeros to a power of two and crop the result. Arithmetic complexity does not guarantee faster small instances.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strassen: form seven products

Partition square matrices into equal blocks $A = [A_{11}, A_{12}; A_{21}, A_{22}]$ and $B = [B_{11}, B_{12}; B_{21}, B_{22}]$. Form seven block products:

$P_1 = (A_{11} + A_{22})(B_{11} + B_{22})$; $P_2 = (A_{21} + A_{22})B_{11}$; $P_3 = A_{11}(B_{12} - B_{22})$; $P_4 = A_{22}(B_{21} - B_{11})$.

$P_5 = (A_{11} + A_{12})B_{22}$; $P_6 = (A_{21} - A_{11})(B_{11} + B_{12})$; $P_7 = (A_{12} - A_{22})(B_{21} + B_{22})$. Block multiplication is not commutative.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strassen: recombination and proof

$C_{11} = P_1 + P_4 - P_5 + P_7$, $C_{12} = P_3 + P_5$; $C_{21} = P_2 + P_4$, $C_{22} = P_1 - P_2 + P_3 + P_6$.

For instance, expanding C_{11} cancels the cross terms and leaves $A_{11}B_{11} + A_{12}B_{21}$. Expanding the other blocks gives ordinary block multiplication. Seven half-size products and $\Theta(n^2)$ additions per level yield $T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7})$; zero-pad to a power of two if needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strassen: a 2×2 trace

Let $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$. The seven scalar products are $P_1 = 65$, $P_2 = 35$, $P_3 = -2$, $P_4 = 8$, $P_5 = 24$, $P_6 = 22$, $P_7 = -30$.

Recombination gives $C_{11} = 65 + 8 - 24 - 30 = 19$, $C_{12} = -2 + 24 = 22$, $C_{21} = 35 + 8 = 43$, and $C_{22} = 65 - 35 - 2 + 22 = 50$, matching direct multiplication. Arithmetic counts alone do not capture finite precision or small-size constants.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Elimination stores multipliers and updates the tail

At step k, use a nonzero pivot u_{kk}, set $l_{ik} = a_{ik}/u_{kk}$, and apply $R_i \leftarrow R_i - l_{ik}R_k$. The surviving upper triangle is U; the multipliers form L. The first block update is the Schur complement $A_{22} - a_{21}a_{12}/a_{11}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An invertible matrix can have a zero pivot

The matrix $A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$ has determinant -1, but elimination without a row exchange divides by zero immediately. With $P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, $PA = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = IU$. A zero chosen pivot alone does not prove that the original matrix is singular.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

First-column row exchange and elimination

Use the CLRS28.1 example $A = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{bmatrix}$. Partial pivoting chooses the largest available absolute entry. Swap $R_1 \leftrightarrow R_3$ to obtain $[5, 6, 3; 3, 4, 4; 1, 2, 0]$. With multipliers $3/5$ and $1/5$, eliminate to obtain $[5, 6, 3; 0, 2/5, 11/5; 0, 4/5, -3/5]$.

Round 1: compare, swap, eliminate

A

$$\begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{bmatrix}$$

P₁A

$$\begin{bmatrix} 5 & 6 & 3 \\ 3 & 4 & 4 \\ 1 & 2 & 0 \end{bmatrix}$$

U after round 1

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 2/5 & 11/5 \\ 0 & 4/5 & -3/5 \end{bmatrix}$$

|1| < |3| < |5| → R₁ ↔ R₃

l₂₁ = 3/5, l₃₁ = 1/5; R₂ ← R₂ − (3/5)R₁; R₃ ← R₃ − (1/5)R₁

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A later swap must move stored multipliers

In column two, $4/5 > 2/5$, so exchange working rows two and three. Also exchange the already stored first-column multipliers in L, giving $l_{21} = 1/5, l_{31} = 3/5$. The working matrix is $[5, 6, 3; 0, 4/5, -3/5; 0, 2/5, 11/5]$. With $l_{32} = 1/2$, the final pivot is $11/5 - (1/2)(-3/5) = 5/2$.

Round 2: swap the working matrix, P and stored L

U₁

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 2/5 & 11/5 \\ 0 & 4/5 & -3/5 \end{bmatrix}$$

P₁

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

L: stored column 1

$$\begin{bmatrix} 1 & 0 & 0 \\ 3/5 & 1 & 0 \\ 1/5 & 0 & 1 \end{bmatrix}$$

|4/5| > |2/5| → R₂ ↔ R₃

Before the swap; exchange only the completed column of L.

Verify the convention with PA=LU

$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$, $L = \begin{bmatrix} 1 & 0 & 0 \\ 1/5 & 1 & 0 \\ 3/5 & 1/2 & 1 \end{bmatrix}$, and $U = \begin{bmatrix} 5 & 6 & 3 \\ 0 & 4/5 & -3/5 \\ 0 & 0 & 5/2 \end{bmatrix}$. Both PA and LU equal $[5, 6, 3; 1, 2, 0; 3, 4, 4]$. P reorders equations, leaving the unknown ordering unchanged.

Eliminate and verify every entry of PA = LU

U after R₂ ↔ R₃

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 4/5 & -3/5 \\ 0 & 2/5 & 11/5 \end{bmatrix}$$

U

$$\begin{bmatrix} 5 & 6 & 3 \\ 0 & 4/5 & -3/5 \\ 0 & 0 & 5/2 \end{bmatrix}$$

PA = LU

$$\begin{bmatrix} 5 & 6 & 3 \\ 1 & 2 & 0 \\ 3 & 4 & 4 \end{bmatrix}$$

l₃₂ = (2/5)/(4/5) = 1/2; R₃ ← R₃ − (1/2)R₂

LU rows: (5, 6, 3); (1, 6/5+4/5, 3/5−3/5);

(3, 18/5+2/5, 9/5−3/10+5/2) = (3, 4, 4)

Why can LUP keep finding a pivot?

Consider an invertible current trailing block K. Its first column contains a nonzero entry; otherwise K would have a zero column and be singular. Swap rows within the trailing block to move such an entry to the pivot a. The swap preserves invertibility.

Write $K' = PK$ for the block after the swap. Block elimination gives $\det K' = a \det S$, while $\det K' = \pm \det K$. Since $\det K \neq 0$ and $a \neq 0$, the new Schur complement S is invertible, and induction continues to the last pivot. Later row swaps also exchange corresponding multipliers already stored in L, producing $PA = LU$. Selecting the largest absolute pivot controls local multipliers but does not guarantee numerical stability for every problem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

In-place LUP: inputs, outputs and storage

Input is a real $n \times n$ matrix A. Start with working array $W = A$ and permutation vector $p = [1, \ldots, n]$. On success, the diagonal and upper triangle of W store U, its strict lower triangle stores the multipliers of L, and the unit diagonal of L is implicit. The convention is $(Pb)_i = b_{p_i}$ and $PA = LU$.

The algorithm below uses exact arithmetic and one-based indices; for $n = 0$ one may return the empty factorization. “In place” means overwriting the working matrix; preserving the original A requires another $\Theta(n^2)$ storage.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

In-place LUP: pivot, swap the whole row, update

For $k = 1, \ldots, n$, choose q maximizing $|W_{ik}|$ among $i = k, \ldots, n$. If $W_{qk} = 0$, report a singular matrix and stop; otherwise exchange entire rows k, q of W, and exchange p_k, p_q.

For each $i = k+1, \ldots, n$, first store $W_{ik} \leftarrow W_{ik}/W_{kk}$; then for $j = k+1, \ldots, n$ update $W_{ij} \leftarrow W_{ij} - W_{ik}W_{kj}$. Swapping whole rows includes earlier multipliers in columns $j < k$. Swapping only the trailing block would break $PA = LU$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Invariant for the completed blocks

After the first k steps, the currently permuted original matrix satisfies

$PA = \begin{bmatrix} L_{11} & 0 \\ L_{21} & I \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} \\ 0 & S \end{bmatrix}$, with $k \times k$ leading triangular blocks and remaining Schur complement S. Completed columns L_{11}, L_{21} and completed rows U_{11}, U_{12} are stored in W.

The next row swap also reorders the already-stored multipliers in L_{21}. Dividing below the new pivot gives the new multipliers, and the outer-product update modifies S exactly as block multiplication requires. Initially $k = 0$ and $S = A$; at $k = n$ the trailing block is empty and $PA = LU$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

What a zero pivot reports

Under exact arithmetic, if all remaining entries in the current pivot column are zero, the trailing block has a zero column and is singular. Earlier pivots were nonzero and all previous permutations and elimination operations were invertible, so the original A is singular. Conversely, an invertible A has an invertible trailing block at each step and hence a nonzero candidate pivot.

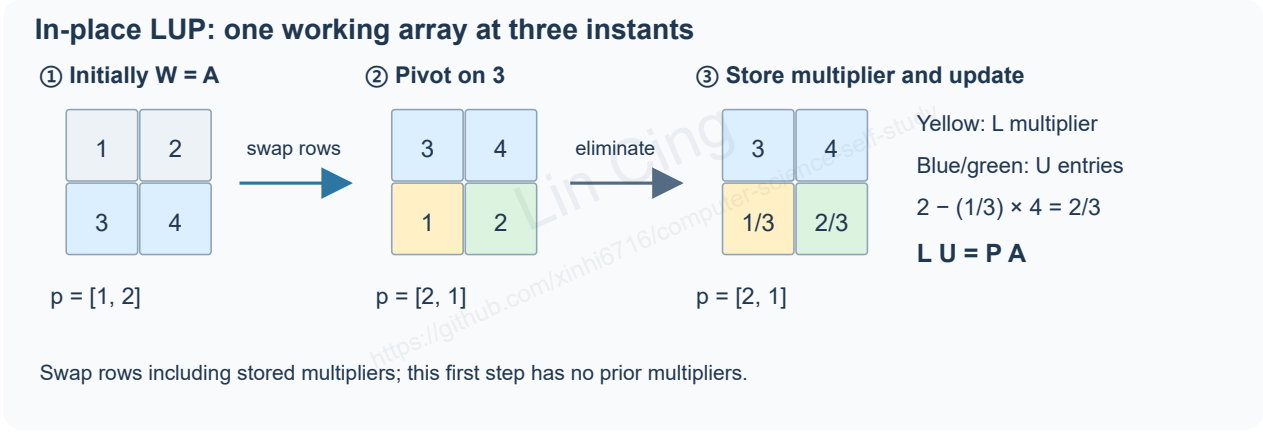
In floating point, treating a small pivot as zero under a chosen scale and tolerance only reports failure under that rule; it does not prove exact mathematical singularity. Rounding, scaling and conditioning matter. Numerical rank requires an appropriate rank-revealing method and an explicit tolerance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Trace the in-place array

Take $A = [1, 2; 3, 4]$ and $b = (5, 11)^T$. The largest absolute value in column 1 is 3. Swap rows and set $p = (2, 1)$: the working matrix becomes $[3, 4; 1, 2]$.

Compute $l_{21} = 1/3$ and update the lower-right entry to $2 - (1/3)4 = 2/3$. Thus $W = [3, 4; 1/3, 2/3]$ stores $L = [1, 0; 1/3, 1]$ and $U = [3, 4; 0, 2/3]$. Check $LU = [3, 4; 1, 2] = PA$ and $Pb = (11, 5)^T$.



Forward substitution computes the intermediate vector

For $b = (3, 7, 8)^T$, permute to $Pb = (8, 3, 7)^T$. Solve $Ly = Pb$. Then $y_1 = 8$, $y_2 = 3 - (1/5)8 = 7/5$, and $y_3 = 7 - (3/5)8 - (1/2)(7/5) = 3/2$. In general, $y_i = (Pb)_i - \sum_{j<i} l_{ij} y_j$. One forward solve takes $\Theta(n^2)$ time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Back substitution starts at the last row

Solve $Ux = y$. Obtain $x_3 = (3/2)/(5/2) = 3/5$, $x_2 = [7/5 + (3/5)(3/5)]/(4/5) = 11/5$, and $x_1 = [8 - 6(11/5) - 3(3/5)]/5 = -7/5$. Substitution gives $Ax = (3, 7, 8)^T$. In general, $x_i = (y_i - \sum_{j>i} u_{ij} x_j)/u_{ii}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Factor once for many right-hand sides

Dense elimination updates $\sum_{k=1}^{n-1}(n-k)^2 = \Theta(n^3)$ trailing entries. With the factors available, each new right-hand side needs two triangular solves in $\Theta(n^2)$. For q right-hand sides, the total is $\Theta(n^3 + qn^2)$. Partial pivoting can still suffer element growth; it does not guarantee accuracy for every problem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Derive determinants and inverses from the factors

After s row exchanges, $\det A = (-1)^s \prod_i u_{ii}$. Two exchanges give $\det A = 5(4/5)(5/2) = 10$. To invert, solve $Az_j = e_j$; z_j is inverse column j. After factorization, n solves cost $\Theta(n^3)$. For a single $Ax = b$, use the factors directly instead of computing the whole inverse.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Forward and back substitution invariants

For unit lower-triangular L, solve $Ly = Pb$ by $y_i = (Pb)_i - \sum_{j<i} l_{ij} y_j$. Before step i, rows $1, \ldots, i-1$ hold and those y_j stay fixed. The formula makes row i hold; induction gives $Ly = Pb$.

For nonsingular upper-triangular U, solve $Ux = y$ from $i = n$ down to 1 using $x_i = (y_i - \sum_{j>i} u_{ij} x_j)/u_{ii}$. Before step i, rows $i+1, \ldots, n$ hold; their equations do not depend on x_i, and the step establishes row i. Nonzero pivots make division valid.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both triangular solves on the same example

Forward substitution gives $y_1 = 11$ and $y_2 = 5 - (1/3)11 = 4/3$. Back substitution gives $x_2 = (4/3)/(2/3) = 2$ and $x_1 = (11 - 4 \cdot 2)/3 = 1$. Finally $A(1, 2)^T = (5, 11)^T = b$.

Row 2 of the upper-triangular system holds first. Computing x_1 leaves x_2 unchanged, so row 2 still holds: this is the back-substitution invariant in a concrete calculation.

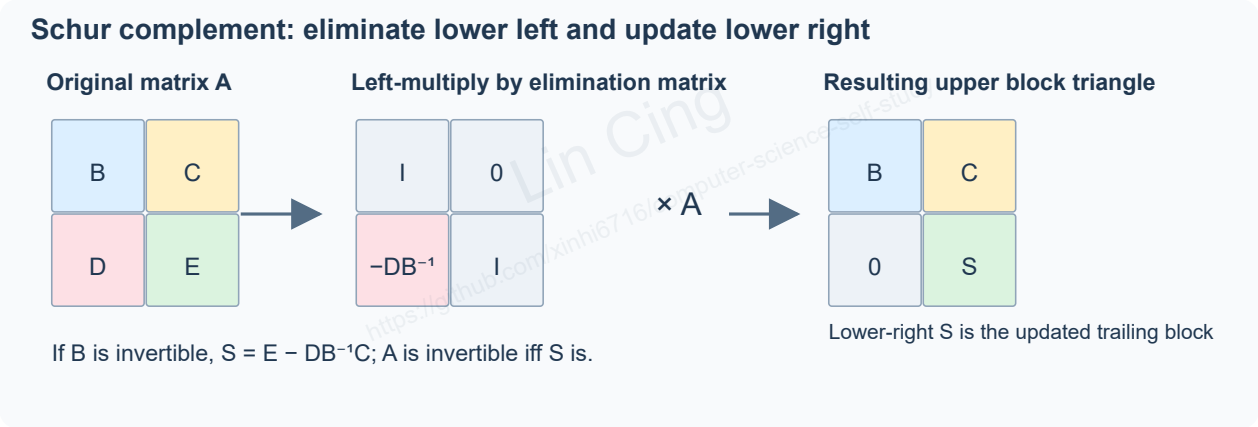
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Block inversion starts with elimination

Let $A = \begin{bmatrix} B & C \\ D & E \end{bmatrix}$, with square, invertible B and compatible dimensions. Its Schur complement is $S = E - DB^{-1}C$. Block elimination gives

$$\begin{bmatrix} I & 0 \\ -DB^{-1} & I \end{bmatrix} A = \begin{bmatrix} B & C \\ 0 & S \end{bmatrix}.$$

Eliminate the lower-left block first, then work on S. Block products cannot be reordered.



Block inverse and its hypotheses

If both B and S are invertible, reversing the block elimination factors yields

$$A^{-1} = \begin{bmatrix} B^{-1} + B^{-1}CS^{-1}DB^{-1} & -B^{-1}CS^{-1} \\ -S^{-1}DB^{-1} & S^{-1} \end{bmatrix}.$$

Check the result by multiplying AA^{-1}. When B is invertible, A is invertible exactly when S is. If B is singular this formula does not apply, even though A itself may be invertible.

A block inverse: compute the Schur complement

Take $A = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 3 \end{bmatrix}$ and partition after the first two rows and columns. Then $B = \begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix}$, $C = (2,1)^T$, $D = C^T$, $E = 3$.

$B^{-1} = \frac{1}{16}\begin{bmatrix} 5 & -2 \\ -2 & 4 \end{bmatrix}$, $B^{-1}C = (1/2, 0)^T$, and $S = 3 - (2,1)(1/2,0)^T = 2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Assemble the inverse block by block

$S^{-1} = 1/2$ and the upper-right block is $-B^{-1}CS^{-1} = (-1/4, 0)^T$; the lower-left block is its transpose.

The upper-left block is $B^{-1} + (1/2,0)^T(1/2)(1/2,0) = \begin{bmatrix} 7/16 & -1/8 \\ -1/8 & 1/4 \end{bmatrix}$.

Thus $A^{-1} = \begin{bmatrix} 7/16 & -1/8 & -1/4 \\ -1/8 & 1/4 & 0 \\ -1/4 & 0 & 1/2 \end{bmatrix}$. For example, the first row times the first column gives $4(7/16) + 2(-1/8) + 2(-1/4) = 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Positive definiteness protects recursive pivots

A real symmetric matrix is positive definite (SPD) when $z^T Az > 0$ for every nonzero z. With $A = \begin{bmatrix} B & C \\ C^T & D \end{bmatrix}$, the principal block B is SPD: choose $z = (x, 0)$ for nonzero x.

The Schur complement $S = D - C^T B^{-1} C$ is also SPD: for nonzero y choose $z = (-B^{-1}Cy, y)$, so $y^T Sy = z^T Az > 0$. Consequently B and S are invertible and the block formula can recurse.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cholesky: square root and trailing update

Every real symmetric positive-definite matrix has a unique factorization $A = LL^T$, where L is lower triangular with positive diagonal. Partition $A = \begin{bmatrix} a & r^T \\ r & D \end{bmatrix}$. Since $a > 0$, set $\ell_{11} = \sqrt{a}$ and $v = r/\sqrt{a}$; the remaining matrix $S = D - vv^T$ is SPD.

Recursively factor $S = L_2 L_2^T$ to obtain $L = \begin{bmatrix} \sqrt{a} & 0 \\ v & L_2 \end{bmatrix}$. Each square root is positive. The first column is forced by the positive diagonal convention, and induction proves uniqueness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Elementwise Cholesky recurrence

For real symmetric positive-definite $A = LL^T$ with positive-diagonal lower-triangular L, the identity $a_{ij} = \sum_{k=1}^{\min(i,j)} l_{ik} l_{jk}$ gives the column-by-column recurrence for $j = 1, \ldots, n$:

$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}$; for $i > j$, $l_{ij} = \dfrac{a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk}}{l_{jj}}$.

An empty sum is zero. Compute the diagonal first, then the entries below it; all columns $k < j$ have already been computed. Entries above the diagonal are zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why are the square roots and divisions valid?

At step j, the radicand is the leading diagonal entry of the Schur complement remaining after eliminating columns $1, \ldots, j-1$. Principal blocks and Schur complements of an SPD matrix are SPD, so this entry is strictly positive. Thus $l_{jj} > 0$ and the division below it is defined.

Substituting the computed entries into $a_{ij} = \sum_{k \le \min(i,j)} l_{ik} l_{jk}$ shows, one column at a time, that $A = LL^T$. The positive diagonal forces each square root and each entry below it uniquely. Without real symmetry and positive definiteness, the radicand or divisor need not be positive.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compute every entry in the 3×3 example

For $A = [4, 2, 2; 2, 5, 1; 2, 1, 3]$, the first column is $l_{11} = \sqrt{4} = 2$, $l_{21} = 2/2 = 1$, $l_{31} = 2/2 = 1$.

The second column is $l_{22} = \sqrt{5 - l_{21}^2} = 2$, $l_{32} = (1 - l_{31}l_{21})/l_{22} = 0$. Finally $l_{33} = \sqrt{3 - l_{31}^2 - l_{32}^2} = \sqrt{2}$.

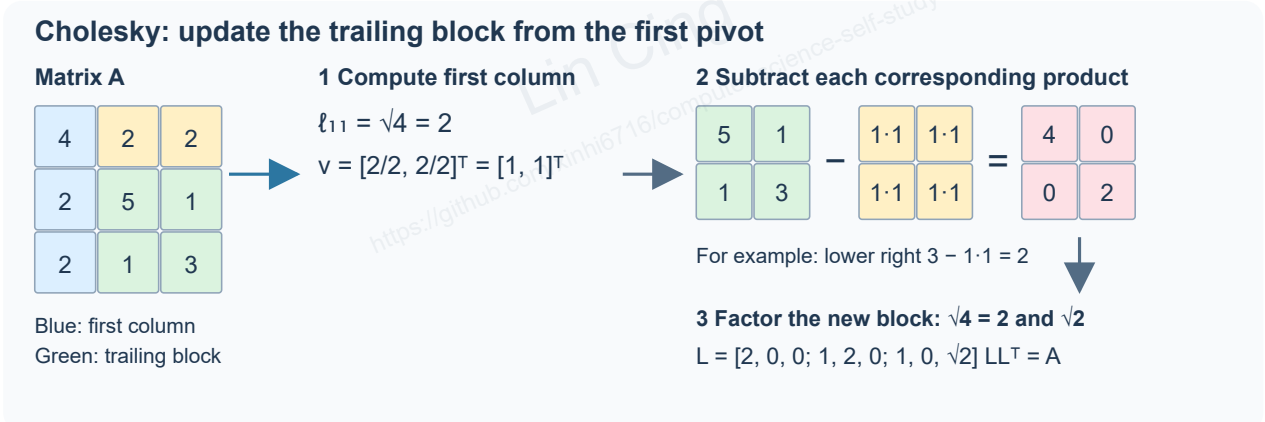
Thus $L = [2, 0, 0; 1, 2, 0; 1, 0, \sqrt{2}]$. Check $l_{31}l_{21} + l_{32}l_{22} = 1 = a_{32}$ and verify every entry with $LL^T = A$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cholesky example: follow the update

For $A = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 3 \end{bmatrix}$, the first pivot is $\ell_{11} = 2$, and both entries below it equal 1. The trailing update is $\begin{bmatrix} 5 & 1 \\ 1 & 3 \end{bmatrix} - \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix}$.

Hence $L = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 0 & \sqrt{2} \end{bmatrix}$. Multiplication gives $LL^T = A$. Also $x^T Ax = \|L^T x\|^2 > 0$ for nonzero x, verifying that this example is SPD.



Solve using a Cholesky factor

Solve $LL^Tx = b$ by forward substitution $Ly = b$ and back substitution $L^Tx = y$. For $b = (8, 8, 6)^T$, forward substitution gives $y = (4, 2, \sqrt{2})^T$. Back substitution yields $x_3 = 1$, $x_2 = 1$, $x_1 = 1$; check $A(1, 1, 1)^T = b$.

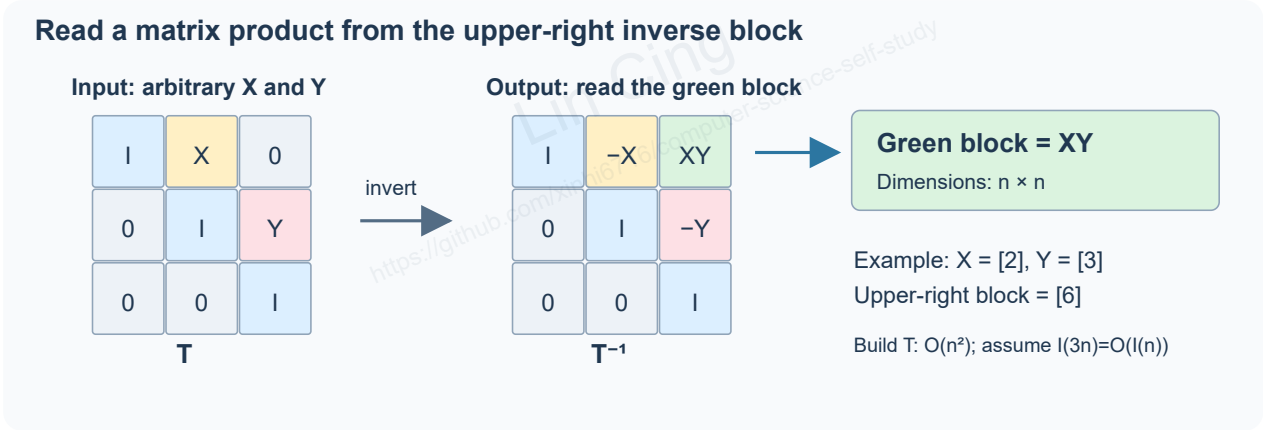
Dense Cholesky factorization costs $\Theta(n^3)$ arithmetic operations; each subsequent right-hand side costs $\Theta(n^2)$ for both triangular solves. The SPD hypothesis matters; near-positive-definite floating-point inputs also need rounding and failure handling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matrix inversion can compute a product

For two $n \times n$ matrices X, Y, form $T = \begin{bmatrix} I & X & 0 \\ 0 & I & Y \\ 0 & 0 & I \end{bmatrix}$.

$T^{-1} = \begin{bmatrix} I & -X & XY \\ 0 & I & -Y \\ 0 & 0 & I \end{bmatrix}$. Block multiplication checks the formula: the upper-right block is $XY - XY = 0$ and the other off-diagonal blocks cancel. Read XY from the upper-right block; T is invertible for every X, Y.



The cost of extracting a product

Let $I(n)$ bound the cost of inverting an invertible $n \times n$ matrix. If $I(n) = \Omega(n^2)$ and $I(3n) = O(I(n))$, forming the $3n \times 3n$ matrix T and extracting its upper-right block takes $O(n^2)$, so matrix multiplication costs $M(n) = O(I(n))$.

This reduction counts arithmetic operations. The scaling assumption matters: a bound for one dimension alone does not justify replacing $I(3n)$ by $I(n)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fast multiplication gives fast inversion

For an SPD matrix, the block formula recursively inverts B and S, and uses a constant number of half-size products and $O(n^2)$ additions. Thus $I_{SPD}(n) \leq 2I_{SPD}(n/2) + O(M(n) + n^2)$.

If $M(n) = \Omega(n^2)$ and $M(n/2) \leq cM(n)$ for a fixed $c < 1/2$, successive recursion levels cost at most a ratio $2c < 1$ of the preceding bound. Summing yields $I_{SPD}(n) = O(M(n))$. The conclusion needs these cost assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

General invertible matrices and the arithmetic model

For real nonsingular A, A^TA is SPD since $x^TA^TAx = \|Ax\|^2 > 0$ for nonzero x. Form and invert A^TA, then multiply by A^T: $A^{-1} = (A^TA)^{-1}A^T$. To pad to a power of two, use $\mathrm{diag}(A, I)$ and assume $M(n+k) = O(M(n))$ for $0 \le k \le n$.

Under the stated regularity conditions and exact arithmetic model, multiplication and inversion reduce to one another. This does not guarantee floating-point stability: forming A^TA squares the condition number, so practical solves typically use pivoted elimination or a suitable factorization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The relation between LDLᵀ and Cholesky

For SPD matrices we may retain the elimination pivots in a diagonal matrix: $A = LDL^T$, where L is unit lower triangular and D is positive diagonal. Each pivot is the top-left entry of the current SPD Schur complement and is therefore positive.

For $D = \mathrm{diag}(d_1, \dots, d_n)$, set $D^{1/2} = \mathrm{diag}(\sqrt{d_i})$. Then $A = (LD^{1/2})(LD^{1/2})^T$, giving the positive-diagonal Cholesky factor. In the previous example $L = [1, 0, 0; 1/2, 1, 0; 1/2, 0, 1]$, $D = \mathrm{diag}(4, 4, 2)$; multiplying by $D^{1/2}$ gives $[2, 0, 0; 1, 2, 0; 1, 0, \sqrt{2}]$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Leading arithmetic cost of dense factorizations

Assume dense $n \times n$ input, count each scalar addition, subtraction, multiplication and division as one flop, and no early singular return. LUP trailing updates contribute about $2\sum_{s=1}^{n-1} s^2 = \frac{2}{3}n^3 + O(n^2)$ flops; multiplier divisions add $O(n^2)$. The two triangular solves for one right-hand side take about $2n^2 + O(n)$ flops together.

For Cholesky, elementwise multiply-add terms take $\frac{1}{3}n^3 + O(n^2)$ flops; divisions and square roots occur $O(n^2)$ and n times respectively. Square roots may have a different cost and should be reported separately. These leading coefficients depend on the stated flop convention and are not wall-clock-time ratios.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Account separately for pivots and memory traffic

Partial pivot selection scans absolute values and makes $O(n^2)$ comparisons. At most n full-row swaps move $O(n^2)$ matrix entries and $O(n)$ permutation indices. These comparisons, memory accesses and square roots are not normally included in the multiply-add flop totals; actual time can be dominated by memory behavior or implementation choices.

W occupies $\Theta(n^2)$ storage and the permutation needs $\Theta(n)$ extra space. Whether the original matrix is retained changes the additional storage. Sparse matrices, low-rank shortcuts, parallelism and early exits are outside the direct dense full-run time accounting.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three noncollinear points require a fit

Fit $b \approx \alpha + \beta t$ to $(t, b) = (0, 1), (1, 2), (2, 2)$. Let $A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \in \mathbb{R}^{3\times 2}$, $x = (\alpha, \beta)^T$, and $b = (1, 2, 2)^T$. Consecutive slopes are 1 and 0, so no line passes through all three points. Minimize $\|b - Ax\|_2^2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Set the gradient to zero for the normal equations

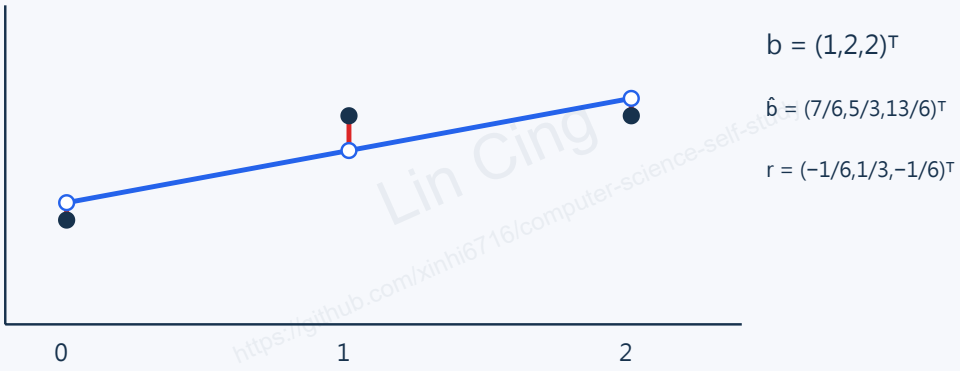
The gradient is $-2A^T(b - Ax)$, so $A^TA\hat{x} = A^Tb$. Here $A^TA = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}$ and $A^Tb = (5, 6)^T$. Full column rank gives $z^TA^TAz = \|Az\|^2 > 0$ for every nonzero z, hence a unique solution. Dependent columns invalidate that uniqueness argument.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compute coefficients, predictions, and residuals

The equations are $3\alpha + 3\beta = 5$ and $3\alpha + 5\beta = 6$. Subtraction gives $\beta = 1/2$, then $\alpha = 7/6$. Predictions are $A\hat{x} = (7/6, 5/3, 13/6)^T$ and residuals are $r = (-1/6, 1/3, -1/6)^T$. The squared error is $1/36 + 1/9 + 1/36 = 1/6$.

Vertical residuals, not shortest point-line distances

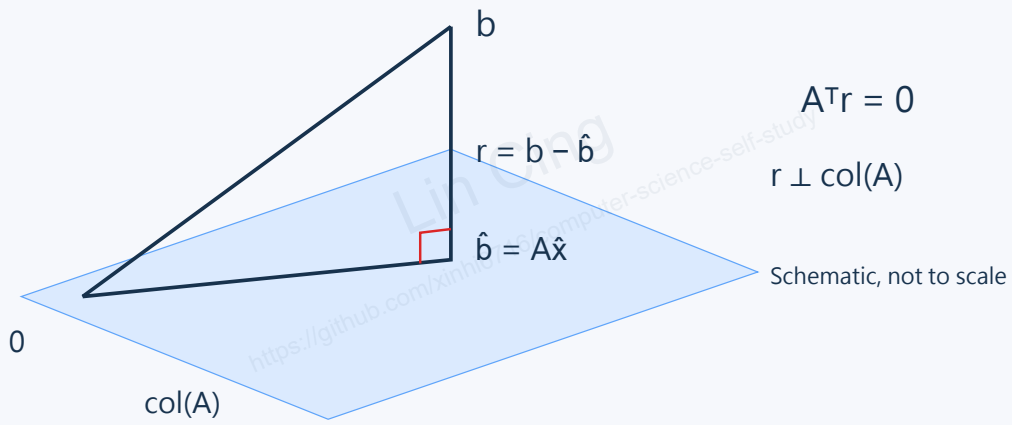


Red segments fix t; blue line b̂ = 7/6 + t/2

Vertical residuals and data-space orthogonality

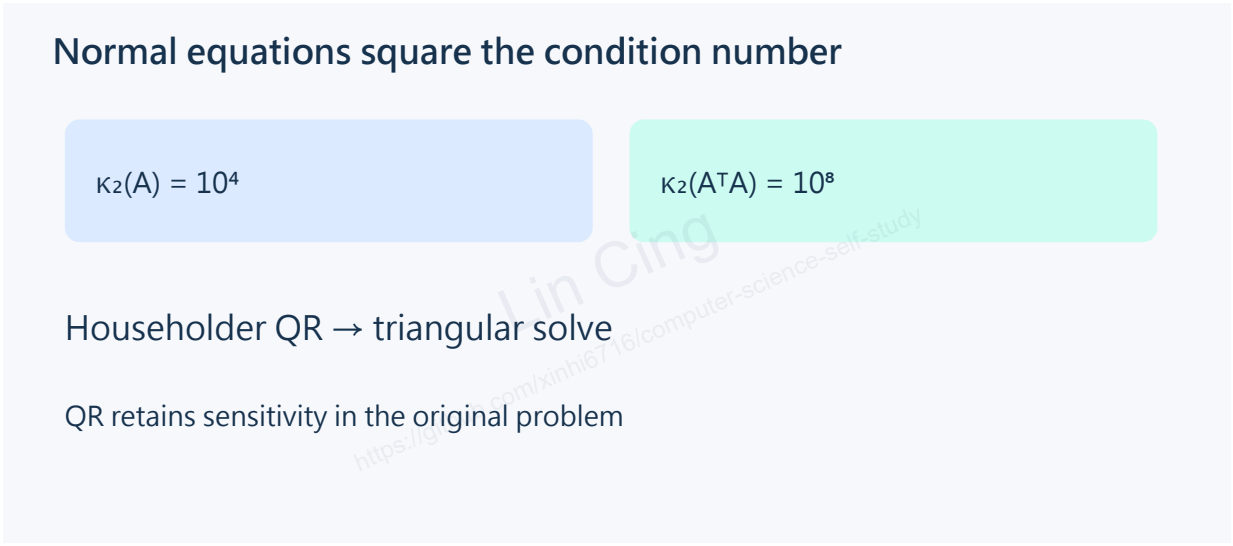
In the scatterplot, draw vertical segments at fixed t_i with length $|b_i - \hat{b}_i|$. These are not shortest distances to the fitted line. In data space $\mathbb{R}^3$, $\hat{b} = A\hat{x} \in \mathrm{col}(A)$ and $r \perp \mathrm{col}(A)$. Check $A^T r = (-1/6 + 1/3 - 1/6,\ 1/3 - 2/6)^T = (0, 0)^T$.

Orthogonal projection in data space R³



Normal equations square the condition number

For full column rank, $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$. Since A^TA has eigenvalues σ_i^2, $\kappa_2(A^TA) = \kappa_2(A)^2$. For $A = \mathrm{diag}(1, 10^{-4})$, the numbers are 10^4 and 10^8. Forming A^TA in floating point makes weak directions harder to retain; there is no universal fixed digit loss.



QR avoids forming the squared-condition matrix

With thin QR, $A = QR$ and $Q^TQ = I$. Solve $R\hat{x} = Q^Tb$. Here $q_1 = (1,1,1)^T/\sqrt{3}$, $q_2 = (-1,0,1)^T/\sqrt{2}$, $R = [\sqrt{3}, \sqrt{3}; 0, \sqrt{2}]$, and $Q^Tb = (5/\sqrt{3}, 1/\sqrt{2})^T$, giving $(7/6, 1/2)^T$ again. Use Householder QR in practice; use pivoted QR or SVD when rank is uncertain. QR does not remove ill-conditioning in the data.

Any rank: the normal equation is necessary

Let $f(x) = \|b - Ax\|_2^2$. If $\hat{x}$ is a global minimizer, then for every direction z, the scalar function $g(t) = f(\hat{x} + tz)$ has a minimum at $t = 0$. Thus $g'(0) = -2z^T A^T (b - A\hat{x}) = 0$.

Because this holds for every z, $A^T(b - A\hat{x}) = 0$, or $A^T A\hat{x} = A^T b$. No full-column-rank or invertibility assumption is needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Any rank: the normal equation is sufficient

Conversely, suppose $A^T(b - A\hat{x}) = 0$ and write $r = b - A\hat{x}$. For any z, expand:

$\|b - A(\hat{x} + z)\|^2 = \|r - Az\|^2 = \|r\|^2 - 2z^T A^T r + \|Az\|^2 = \|r\|^2 + \|Az\|^2 \geq \|r\|^2$.

Therefore $\hat{x}$ is a global minimizer. Equality holds exactly when $Az = 0$, so all minimizers are $\hat{x} + \ker(A)$. Moreover $A\hat{x} \in \mathrm{col}(A)$ and $r \perp \mathrm{col}(A)$, making it the unique orthogonal projection of b. The projection onto a finite-dimensional column space exists, hence at least one such $\hat{x}$ exists.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The projection matrix in the full-column-rank case

When $A \in \mathbb{R}^{m \times n}$ has full column rank, $A^T A$ is SPD and $\hat{x} = (A^T A)^{-1} A^T b$ is the unique least-squares solution. Define $P = A(A^T A)^{-1} A^T$; the fitted vector is $p = Pb$.

$P^T = P$ and $P^2 = P$, so P is the orthogonal projector onto $\mathrm{col}(A)$. Equivalently, $A^T(b - Pb) = 0$ proves the residual is orthogonal. Without full column rank $(A^T A)^{-1}$ does not exist; use the SVD/pseudoinverse formulation instead.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rank deficiency: the fitted vector is unique

For any real A and b, $\min_x \|Ax - b\|_2^2$ has a solution, and all solutions satisfy $A^T Ax = A^T b$. The orthogonal projection p of b onto $\mathrm{col}(A)$ exists and is unique; every x satisfying $Ax = p$ is a minimizer.

If the columns are dependent, $\ker(A)$ contains a nonzero vector. For one solution x_0, all solutions are precisely $x_0 + \ker(A)$: the fit $Ax = p$ stays unique while the coefficients need not. If $A = 0$, every x gives the same residual.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A rank-deficient example

Take $A = \begin{bmatrix} 1 & 1 \\ 2 & 2 \\ 0 & 0 \end{bmatrix}$, $b = (2, 3, 1)^T$, and $u = (1, 2, 0)^T$. Then $p = u(u^T b)/(u^T u) = (8/5, 16/5, 0)^T$ and $r = b - p = (2/5, -1/5, 1)^T$; check $A^T r = 0$.

$Ax = p$ means $x_1 + x_2 = 8/5$, so all solutions are $(4/5, 4/5)^T + t(1, -1)^T$. All give the same fit. The minimum-norm coefficient vector occurs at $t = 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SVD selects the unique minimum-norm solution

Let $A = U\Sigma V^T$ have nonzero singular values $\sigma_1, \ldots, \sigma_r$ and define $A^+ = \sum_{i=1}^{r} \sigma_i^{-1} v_i u_i^T$. Then $x_* = A^+ b$ is the unique minimum-norm least-squares solution; all least-squares solutions are $x_* + z$ for $z \in \ker(A)$.

Since $x_* \in \ker(A)^\perp$, $\|x_* + z\|^2 = \|x_*\|^2 + \|z\|^2$; only $z = 0$ minimizes the norm. At rank zero the empty sum gives $A^+ = 0$. Numerical implementations need a scale-aware rank cutoff.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why evaluate polynomials?

For coefficient vectors of lengths m and n, direct multiplication takes $O(mn)$ arithmetic operations. Evaluation at shared points turns multiplication into pointwise multiplication; interpolation returns coefficients.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Can point values determine the coefficients?

For n points $x_0, \ldots, x_{n-1}$ in a field F, coefficient vector a and value vector y satisfy $y = Va$, where $V_{ij} = x_i^j$ for $0 \le i, j < n$. A square matrix alone does not guarantee a unique solution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The Vandermonde determinant

$\det V = \prod_{0 \le i < j < n} (x_j - x_i)$. Thus V is invertible over a field exactly when the points are pairwise distinct. Repeated points give equal rows.

For $1, 2, 4$, the determinant is $(2-1)(4-1)(4-2) = 6$. It is nonzero over the reals but zero over $\mathbb{F}_3$, where 1 and 4 coincide.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Proof: treat the last point as a variable

Proceed by induction. For $n = 1$, the determinant and empty product both equal 1. Treat $x_0, \ldots, x_{n-2}$ as distinct indeterminates, and let $D(t)$ be the determinant whose last row is $[1, t, \ldots, t^{n-1}]$.

D has degree at most $n - 1$. Each substitution $t = x_i$ duplicates a row, so $D(x_i) = 0$. In $F[x_0, \ldots, x_{n-2}, t]$, these distinct monic linear factors are irreducible, so their product divides $D(t)$ and $D(t) = c \prod_{i=0}^{n-2} (t - x_i)$. The resulting polynomial identity can then be specialized to any field elements, including repeated points.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Proof: identify the leading coefficient

Expand along the last row. The only term containing t^{n-1} has positive cofactor equal to the Vandermonde determinant for the first $n-1$ points.

Thus $c = \det V_{n-1} = \prod_{0 \le i < j < n-1} (x_j - x_i)$. Substitute $t = x_{n-1}$ to complete the induction. No distinctness assumption was needed in this identity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lagrange interpolation constructs the solution

For pairwise distinct $x_0, \dots, x_{n-1}$ define $L_i(z) = \prod_{j \ne i} \frac{z - x_j}{x_i - x_j}$. Each denominator is nonzero and invertible in the field. Thus $L_i(x_k) = 1$ when $i = k$ and 0 otherwise.

For arbitrary values y_i, the polynomial $P(z) = \sum_i y_i L_i(z)$ has degree less than n and $P(x_i) = y_i$. If another degree-less-than-n polynomial Q has the same values, $P - Q$ has n distinct roots and must be zero. The interpolant is unique, in agreement with invertibility of the Vandermonde matrix.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Computing interpolating coefficients in $O(n^2)$

Construct $G(z) = \prod_{j=0}^{n-1}(z - x_j)$ by multiplying one linear factor at a time. Updating the coefficient array of length $k+1$ at step k takes $O(k)$, hence $O(n^2)$ field operations overall.

For each i, synthetic division computes $Q_i(z) = G(z)/(z - x_i)$ in $O(n)$. Horner evaluation gives $d_i = Q_i(x_i) = \prod_{j \neq i}(x_i - x_j) \neq 0$. Add the n coefficients of $y_i d_i^{-1} Q_i$ to the answer. The n iterations each cost $O(n)$, so the total is $O(n^2)$ field operations, under the unit-cost field-arithmetic model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fewer than n points do not ensure uniqueness

Given $m < n$ distinct points $x_0, \ldots, x_{m-1}$, both $P(z) = 0$ and $Q(z) = \prod_{i=0}^{m-1}(z - x_i)$ vanish at all the points. They are different polynomials, both of degree less than n.

For example, to determine a polynomial of degree at most two, knowing only $P(0) = P(1) = 0$ admits both $P(z) = 0$ and $P(z) = z(z-1)$. A value at a third distinct point permits the earlier n-point uniqueness theorem to apply.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Horner evaluation and its loop invariant

For $A(z) = \sum_{j=0}^{n-1} a_j z^j$, fix an evaluation point x, set $b = 0$, and update $b \leftarrow a_k + xb$ for $k = n-1, n-2, \ldots, 0$. After processing k, the invariant is $b = \sum_{j=k}^{n-1} a_j x^{j-k}$. Initialization is the empty sum; each update multiplies the prior sum by x and adds a_k. At termination $k = 0$, so $b = A(x)$.

For $A(z) = 2 + 3z + 4z^2$ and $x = 2$, the successive values are 4, $3 + 2 \cdot 4 = 11$, and $2 + 2 \cdot 11 = 24$. This implementation takes $\Theta(n)$ field operations for n coefficients. Repeating it for n points costs $\Theta(n^2)$ for this method; this is not a lower bound for all multipoint evaluation methods.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Set the transform convention

For $n = 2^s$, let $\omega_n = e^{2\pi i/n}$ and $X_k = \sum_{j=0}^{n-1} x_j \omega_n^{jk}$. This is the positive-exponent DFT; indices range from 0 to $n-1$.

$\omega_4 = i$

k: 0 1 2 3

ω_4^k: 1 i −1 −i

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Roots and periodicity

$\omega_n^n = 1$, $\omega_n^{k+n} = \omega_n^k$, and $\omega_n^{k+n/2} = -\omega_n^k$ for even n. For $n = 4$, the roots are $1, i, -1, -i$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Split even and odd coefficients

Write $A(z) = E(z^2) + zO(z^2)$ where $E(t) = \sum_r x_{2r} t^r$ and $O(t) = \sum_r x_{2r+1} t^r$. The two half-size DFTs reuse the same squared roots.

A(z) = E(z²) + z O(z²)

E: x₀, x₂, x₄, …

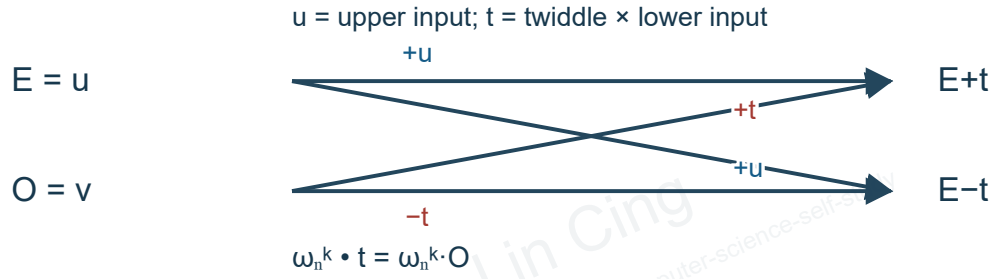
O: x₁, x₃, x₅, …

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Combine a pair

For $0 \le k < n/2$, set $t = \omega_n^k O_k$. Then $X_k = E_k + t$ and $X_{k+n/2} = E_k - t$. The pair is a butterfly; the multiplier is the twiddle factor.

One butterfly



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why the cost is $O(n \log n)$

The recurrence $T(n) = 2T(n/2) + O(n)$ has $\log_2 n$ levels and $O(n)$ work per level. The iterative in-place version has $n/2$ butterflies per stage and $\log_2 n$ stages.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bit-reverse the input

For $n = 4$, reverse the two-bit indices: $00, 01, 10, 11 \mapsto 00, 10, 01, 11$. Thus $[1, 2, 0, -1]$ becomes $[1, 0, 2, -1]$. The permutation prepares the iterative decimation-in-time stages.

index: 00 01 10 11

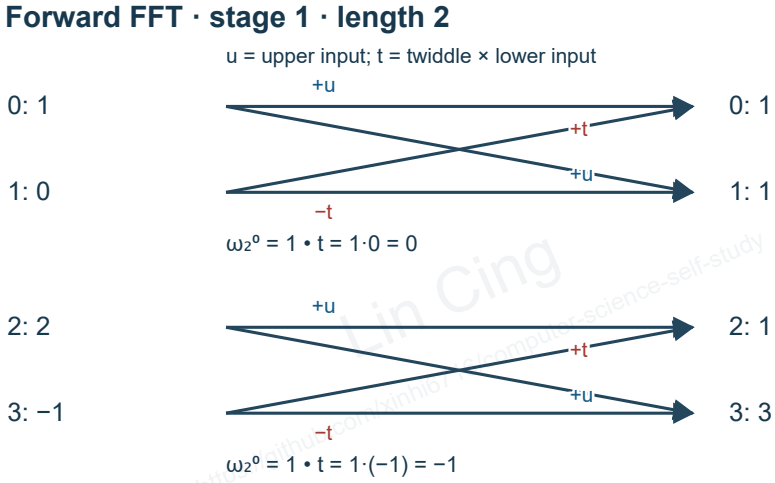
reverse: 00 10 01 11

values: 1 0 2 −1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

First stage, length 2

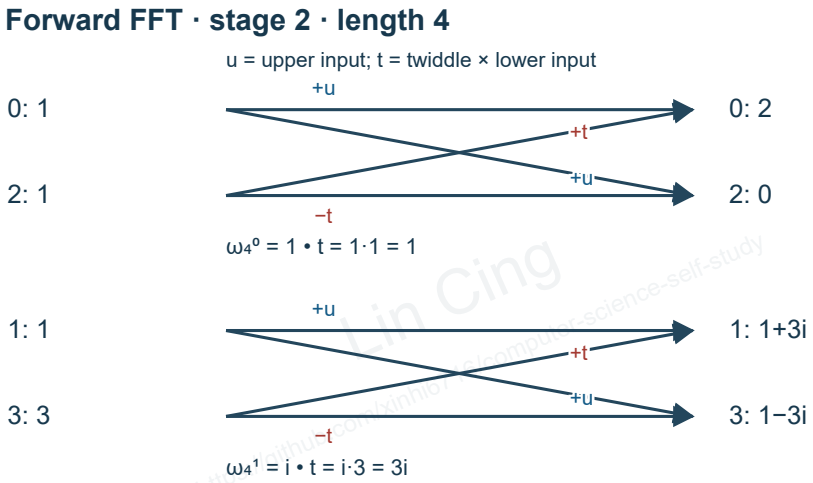
For $[1, 0, 2, -1]$, both twiddles are $\omega_2^0 = 1$. Indices $(0, 1)$ give $(1 + 0, 1 - 0) = (1, 1)$; indices $(2, 3)$ give $(2 - 1, 2 - (-1)) = (1, 3)$. State: $[1, 1, 1, 3]$.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Second stage, length 4

Input $[1, 1, 1, 3]$. Pair $(0, 2)$ uses $\omega_4^0 = 1$: $(1 + 1, 1 - 1) = (2, 0)$. Pair $(1, 3)$ uses $\omega_4^1 = i$: $(1 + 3i, 1 - 3i)$. In index order $X = [2, 1 + 3i, 0, 1 - 3i]$.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

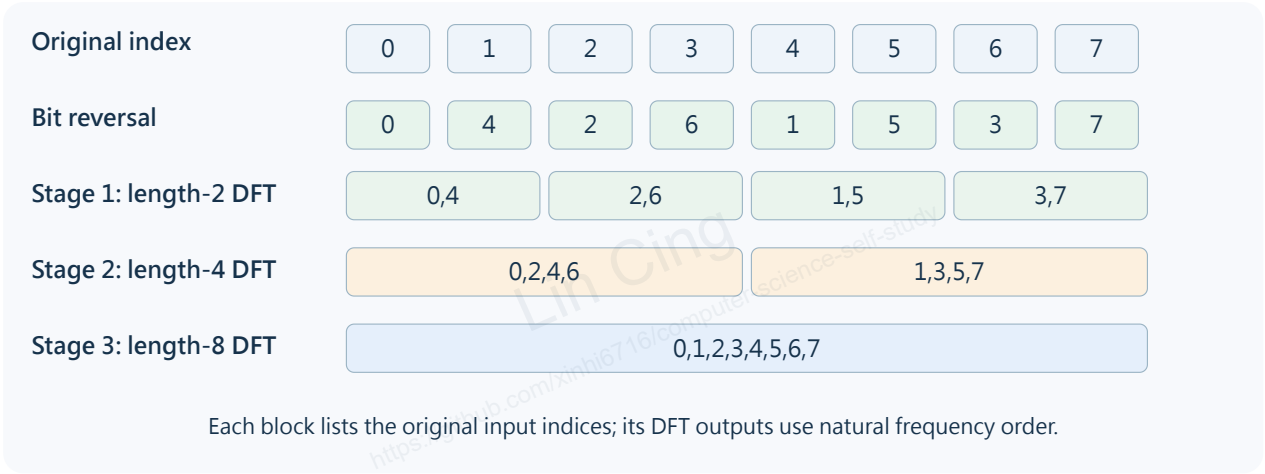
Check with the definition

Direct sums give $X_0 = 1 + 2 + 0 - 1 = 2$, $X_1 = 1 + 2i + (-1)(-i) = 1 + 3i$, $X_2 = 1 - 2 + 0 + 1 = 0$, and $X_3 = 1 - 2i + (-1)i = 1 - 3i$. This independently checks indexing and signs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The iterative butterfly invariant

Let $N = 2^\ell$. After stage s ($0 \le s \le \ell$), each contiguous block of length 2^s stores a length-2^s DFT in natural frequency order. Block b is the DFT of the original subsequence $x_{r+tN/2^s}$, $0 \le t < 2^s$, where r is the reversal of the $\ell - s$ bits of b.



Each block lists the original input indices; its DFT outputs use natural frequency order.

Why each butterfly preserves the invariant

At $s = 0$, each singleton is in bit-reversed order and the claim holds. Assume it holds at stage s: the two halves of each next-stage block are DFTs of the even and odd positions of one larger subsequence. For each k, combine them as $E_k \pm \omega_{2^{s+1}}^k O_k$ in positions k and $k + 2^s$; the result is the DFT of that larger subsequence.

The old subsequences have stride $N/2^s$ and their starting indices differ by $N/2^{s+1}$, exactly the even/odd split of the new subsequence. Induction to $s = \ell$ gives the entire input DFT in natural frequency order.

Bit reversal has a cost

For $N = 2^\ell$, let $\mathrm{rev}_\ell(i)$ reverse the ℓ bits of i. Scanning all ℓ bits independently for every index takes $O(N \log N)$ work. A reversed counter updates only the bits affected by a carry; over all N updates these flips total $O(N)$, so all indices can be generated in $O(N)$ work.

For in-place permutation, swap only when $i < \mathrm{rev}_\ell(i)$ so each pair is swapped once. The earlier parallel FFT assumes the map is supplied when stating $O(1)$ permutation depth; generating it must otherwise be charged separately. Word size and instruction model affect the bit-operation cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The iterative algorithm

Bit-reverse x. For $L = 2, 4, \ldots, n$, for each block beginning at b and $0 \le j < L/2$, set $u = a_{b+j}$ and $t = \omega_L^j a_{b+j+L/2}$; assign $(a_{b+j}, a_{b+j+L/2}) = (u + t, u - t)$. Save u and t before writing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Invert the DFT

The inverse is $x_j = \frac{1}{n}\sum_{k=0}^{n-1} X_k \omega_n^{-jk}$. Apply the same FFT with inverse roots, then divide each result by n.

bit reverse → inverse-root FFT → divide by n

[2, 1+3i, 0, 1−3i] → [1, 2, 0, −1]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

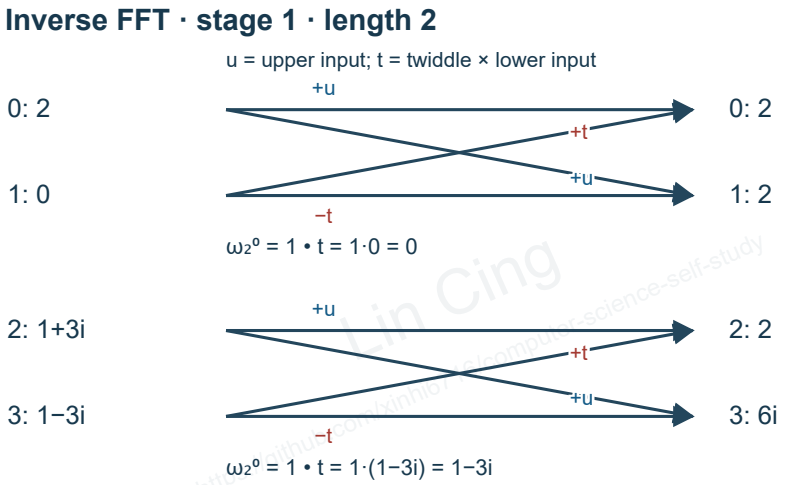
Why division by n works

Substitute X_k: $\frac{1}{n}\sum_r x_r \sum_k \omega_n^{k(r-j)}$. The inner geometric sum is n when $r = j$ and 0 otherwise because $0 \le r, j < n$. Hence only x_j remains.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

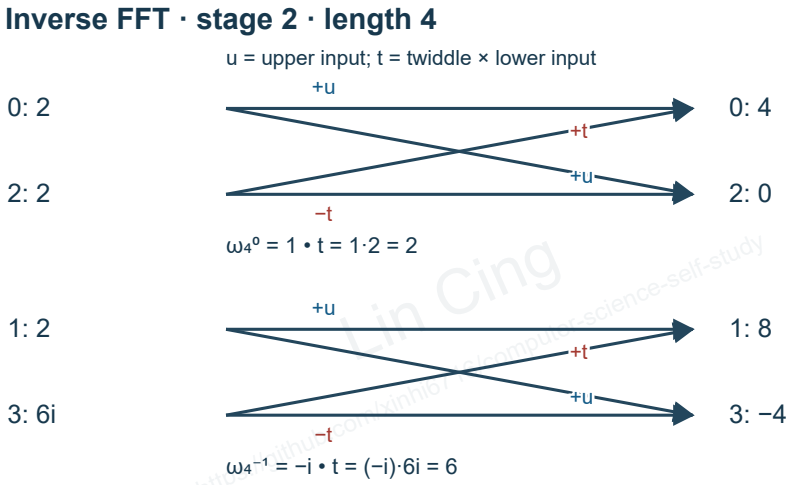
Inverse example, first stage

Bit-reverse $[2, 1+3i, 0, 1-3i]$ to $[2, 0, 1+3i, 1-3i]$. With length-2 twiddle 1, the pairs produce $[2, 2, 2, 6i]$.



Inverse example, second stage

Use $\omega_4^{-1} = -i$. Pair $(0, 2)$ yields $(4, 0)$ with twiddle 1. Pair $(1, 3)$ has $(-i)(6i) = 6$ and yields $(8, -4)$. Divide $[4, 8, 0, -4]$ by 4 to recover $[1, 2, 0, -1]$.



Recursive FFT space: count what is live at once

In a sequential depth-first implementation, suppose each level allocates temporary arrays proportional to its subproblem and releases them when finished. Along one recursion path the live capacity is at most $N + N/2 + N/4 + \cdots < 2N$; including input/output arrays and an $O(\log N)$ call stack gives $O(N)$ peak extra space.

If instead all arrays at every level are retained for tracing or reuse, each level holds N elements and there are $\log_2 N + 1$ levels, so storage is $\Theta(N \log N)$. Both approaches may perform $\Theta(N \log N)$ work; retention policy, not recursion depth alone, accounts for the space difference.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why applying the forward DFT twice is not inversion

Under $X_k = \sum_j x_j \omega^{jk}$, applying the same forward transform again and dividing by N gives $\frac{1}{N} \sum_k X_k \omega^{k\ell} = \frac{1}{N} \sum_j x_j \sum_k \omega^{k(j+\ell)} = x_{(-\ell) \bmod N}$.

The inner sum is N only when $j + \ell \equiv 0 \pmod N$ and is zero otherwise. Thus the result reverses cyclic indices. The inverse instead uses ω^{-1} and divides by N, or follows the second forward transform with index reversal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A concrete counterexample

Let $N = 4, \omega = i$ and $x = [1, 2, 0, 0]$. The first forward DFT is $[3, 1 + 2i, -1, 1 - 2i]$. Applying the same forward root a second time and dividing by 4 produces $[1, 0, 0, 2]$, not the input $[1, 2, 0, 0]$. The issue is the direction of the exponent, not a missing factor of N; using inverse root $-i$ restores the input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Work and depth of parallel FFT

Use a synchronous random-access EREW PRAM, with $N = 2^s \geq 2$ and a given bit-reversal address map. Assign $N/2$ processors per stage. Each processor reads one disjoint input pair, computes one butterfly and writes the pair back; distinct butterflies access disjoint locations. A barrier separates stages. Assume twiddle factors are precomputed.

Bit-reversal copying takes $O(N)$ work and $O(1)$ time; each of $\log_2 N$ stages takes $\Theta(N)$ work and $\Theta(1)$ time. Therefore $W = \Theta(N \log N)$, depth $D = \Theta(\log N)$ and extra storage $O(N)$. This is a model result, not a promise for every machine. Charge index or twiddle generation separately if they are not supplied.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linear versus circular convolution

Pointwise multiplying length-N transforms gives circular convolution $c_k = \sum_{j=0}^{N-1} a_j b_{(k-j) \bmod N}$. Without enough padding, coefficients wrap and add to earlier indices.

m + n − 1 ≤ N

[1, 2, 0, 0] × [3, 4, 0, 0]

linear output: [3, 10, 8, 0]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pad before multiplying

For lengths $m, n \geq 1$, choose a power of two $N \geq m + n - 1$, append zeros to both inputs, FFT, multiply pointwise, and inverse FFT. The first $m + n - 1$ outputs are the linear convolution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Convolution example

$[1,2] * [3,4] = [3,10,8]$. With $N = 4$, pad to $[1,2,0,0]$ and $[3,4,0,0]$. The inverse pointwise product gives $[3,10,8,0]$; $N = 2$ would instead wrap 8 into the constant term.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complex FFT convolution: two forward transforms

For $A(z) = 1 + 2z$, $B(z) = 3 + 4z$, the linear convolution has length $2 + 2 - 1 = 3$, so pad to $N = 4$. Use the positive-exponent convention $\omega = i$: $A = [1,2,0,0]$, $B = [3,4,0,0]$.

Evaluate $X_k = \sum_j x_j i^{jk}$ at $1, i, -1, -i$: $\widehat{A} = [3, 1+2i, -1, 1-2i]$ and $\widehat{B} = [7, 3+4i, -1, 3-4i]$. For example, $A(i) = 1 + 2i$ and $B(-1) = 3 - 4 = -1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multiply matching frequencies and invert

The pointwise products are $C = [21, -5+10i, 1, -5-10i]$; for example $(1+2i)(3+4i) = -5+10i$. The inverse uses $i^{-1} = -i$: $c_j = \frac{1}{4}\sum_{k=0}^{3} C_k(-i)^{jk}$.

The four unscaled sums are $[12, 40, 32, 0]$, so $c = [3, 10, 8, 0]$. Direct multiplication confirms $(1+2z)(3+4z) = 3+10z+8z^2$. Pair values at the same frequency, not across frequencies.

Frequency	Value of A	Value of B		Pointwise product
1	3	7	×	21
i	1+2i	3+4i	×	−5+10i
−1	−1	−1	×	1
−i	1−2i	3−4i	×	−5−10i

Inverse: [21, −5+10i, 1, −5−10i] → [3, 10, 8, 0]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fourier transforms over a finite field

Choose a prime p, let $N = 2^s$ and $N \mid (p-1)$. Choose $\omega \in \mathbb{F}_p$ of multiplicative order exactly N: $\omega^N = 1$ and $\omega^k \neq 1$ for $1 \le k < N$. Such a root exists because the nonzero elements form a cyclic group of order $p-1$.

Define $X_k = \sum_{j=0}^{N-1} x_j \omega^{jk} \pmod{p}$; all arithmetic takes place in $\mathbb{F}_p$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why the inverse works

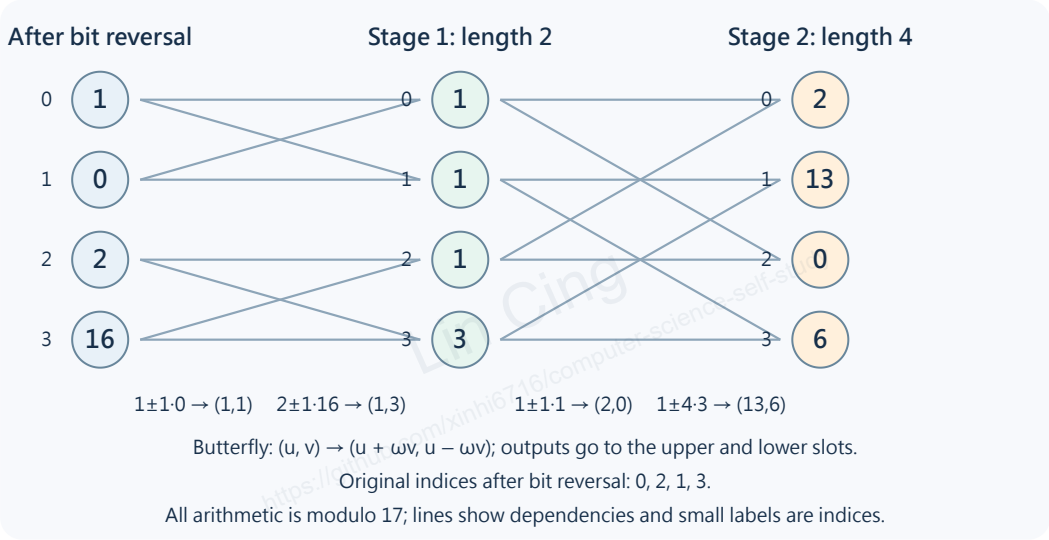
Since $N \mid p-1$, $p \nmid N$, so N^{-1} exists. If $0 < r < N$, then $\omega^r \neq 1$ and $1-\omega^r$ is invertible; consequently $\sum_{k=0}^{N-1} \omega^{kr} = 0$. For $r = 0$, the sum is N.

Substitution and exchange of sums give $x_j = N^{-1} \sum_{k=0}^{N-1} X_k \omega^{-jk} \pmod{p}$. The proof uses invertibility of nonzero field elements; it does not automatically extend to composite moduli.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NTT example: two butterfly stages modulo 17

Take $N = 4$, $p = 17$, $\omega = 4$. Since $4^2 = 16 = -1$ and $4^4 = 1$, its order is exactly 4; $4^{-1} = 13$. Input $[1, 2, 0, -1]$ is $[1, 2, 0, 16]$ modulo 17.



Direct evaluation also gives $[2, 13, 0, 6]$. Interpret every negative value modulo 17.

Pointwise product and integer recovery

Pad $A = [1, 2]$ and $B = [3, 4]$ to $N = 4$. Their NTTs are $[3, 9, 16, 10]$ and $[7, 2, 16, 4]$; their pointwise product is $[4, 1, 1, 6]$. The inverse is $[3, 10, 8, 0]$, giving $3 + 10z + 8z^2$.

Linear rather than cyclic convolution requires $N \geq r + s - 1$. A modular result identifies only a residue. If every integer coefficient has known absolute bound M and $p > 2M$, its symmetric representative is unique. Here $r = s = 2$, $|a_i|, |b_i| \leq 4$ gives the coarse bound $M = 2 \cdot 4^2 = 32$, but $17 \not> 64$; the residue 10 also represents -7, so without a sufficiently tight range bound the integer coefficient cannot be identified uniquely.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Signed integer convolution with a sufficient bound

Take $A = [1, 2]$ and $B = [1, -1]$, again with $p = 17$, $N = 4$, $\omega = 4$. Their padded transforms are $[3, 9, 16, 10]$ and $[0, 14, 2, 5]$. The pointwise product $[0, 7, 15, 16]$ inverses to $[1, 1, 15, 0]$.

Both inputs have coefficient magnitude at most $B = 2$; each convolution coefficient contains at most $\min(2, 2) = 2$ products, so $M = 2B^2 = 8$. Since $p = 17 > 2M = 16$, the symmetric representatives in $[-8, 8]$ are unique: residue 15 decodes to -2, giving $[1, 1, -2, 0]$.

A length-N FFT uses $\Theta(N \log N)$ arithmetic operations modulo p. If p grows with the input, the bit cost of modular additions and multiplications depends on the arithmetic method for $\log p$-bit integers; it is not automatically constant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Precision and cost

Three transforms and N pointwise products take $O(N \log N)$ arithmetic and $O(N)$ storage. This is an arithmetic-operation bound, not a bit-complexity guarantee. Floating-point roundoff accumulates; rounding to integer coefficients is valid only when every absolute error is proved below $1/2$. Otherwise use higher precision or an exact method.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Choose the right method

For small inputs direct multiplication may be simpler. FFT requires suitable padding and a numerical error budget; for exact integer results, consider number-theoretic transforms with suitable moduli and reconstruction bounds. Check convention, indices, and normalization at interfaces.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

CLRS, Introduction to Algorithms, third edition, chapters 28 and 30. Matrix: PDF pages 834–861. FFT: PDF pages 919, 921, 923, 928–936, 939–941, 944–945 (PDF page numbering, third edition). MIT 6.046J Fall 2004 lecture 3 slides, PDF pages 27 and 35–43, and Fall 2005 lecture 3 notes support matrix multiplication and Strassen, not FFT.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study